Reviewer Pack — Minimal Rank of Formal Groups and Read-Twice BP Complexity

Entry 729cd32dd1f6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 04:59:35 UTC

Minimal Rank of Formal Groups and Read-Twice BP Complexity

Entry ID: `729cd32dd1f6` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-22 04:59:35 UTC

1. Conjecture

Field A (mathematical branch): Formal Group Theory **Field B** (complexity object): Complexity Theory: Read-Twice Branching Program Complexity

Statement:

> {'sentence1': 'For any given Boolean function f, the minimal rank of its associated formal group is linearly related to the complexity of computing f using a read-twice branching program.', 'sentence2': 'Specifically, if f can be computed by a read-twice BP with size S(n), then the minimal rank of the formal group representing f is Θ(S(n)).', 'sentence3': 'This relationship holds for all instances of size n ≤ 40.'}

Rationale (proposer's reasoning):

> {'sentence1': 'Formal groups provide a way to encode algebraic structures that can be used to represent Boolean functions, and their ranks could potentially reflect the complexity of computation.', 'sentence2': 'Read-twice BP is a known model for studying circuit complexity, and understanding its relationship with formal group theory might uncover new insights into circuit lower bounds.', 'sentence3': 'The bridge between these fields may reveal novel algebraic structures that are closely tied to computational complexity.'}

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `0239f549222bcf7a`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a given Boolean function of size n ≤ 40, the minimal rank of its associated formal group is considered supported if the correlation coefficient between the formal group ranks and the read-twice BP sizes is ≥ 0.8 AND the mean difference between the ranks and the corresponding BP sizes is ≤ 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `formal group theory AND read-twice BP complexity` - `minimal rank formal group AND complexity of computing via read-twice BP` - `θ(S(n)) minimal rank AND associated with Boolean function`

Top relevant hits considered: - [http://arxiv.org/abs/2309.16111v2] The relational complexity of linear groups acting on subspaces - [http://arxiv.org/abs/hep-ph/0610012v1] Tevatron-for-LHC Report of the QCD Working Group - [http://arxiv.org/abs/2303.14697v2] The central tree property and algorithmic problems on subgroups of free groups - [http://arxiv.org/abs/2501.16039v4] Complexity of Constructing Minimal Faithful Permutation Representations for Fitting-free Groups - [http://arxiv.org/abs/2510.17487v1] Directional Search for Persistent Gravitational Waves: Results from the First Part of LIGO-Virgo-KAGRA's Fourth Observin - [http://arxiv.org/abs/2603.14168v1] All-sky Searches for Continuous Gravitational Waves from Isolated Neutron Stars in the Data from the First Part of the F - [http://arxiv.org/abs/2604.05701v1] Measurement of the CKM angle γ in $B^{\pm} \to D(\to K^0_S h'^+ h'^-) h^{\pm}$ dec - [s2:6b1f52c5d6b9ff86002279c364a925cc99b4b7b3] Intuition Versus Analysis- Which Process is Most Appropriate for Solving Everyday Problems with Differing Levels of Soci

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_boolean_function(n):
    return [random.choice([0, 1]) for _ in range(2**n)]

def formal_group_rank(f):
    n = int(math.log2(len(f)))
    A = [[0] * (n + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        for j in range(n + 1):
            if i == 0 and j == 0:
                A[i][j] = 1
            elif i == 0 or j == 0:
                A[i][j] = 0
            else:
```

```
                A[i][j] = f[2**(i-1) + 2**(j-1)]
    rank = 0
    for row in A:
        if any(row):
            rank += 1
    return rank

def read_twice_bp_size(f):
    n = int(math.log2(len(f)))
    size = 1
    for i in range(n + 1):
        if f[2**i]:
            size += 1
    return size

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for n in [5, 10, 15, 20, 30, 40]:
        for _ in range(5):  # Ensure at least 5 instances per size
            f = generate_boolean_function(n)
            rank = formal_group_rank(f)
            size = read_twice_bp_size(f)
            results.append((rank, size))
    if not results:
        return {
            "metric_name": "correlation_coefficient",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "empty_results"
        }

    ranks = [r for r, _ in results]
    sizes = [s for _, s in results]
    mean_rank = sum(ranks) / len(ranks)
    mean_size = sum(sizes) / len(sizes)
    correlation_coefficient = sum((r - mean_rank) * (s - mean_size) for r, s in results) / (len(re
    mean_difference = abs(mean_rank - mean_size)

    return {
        "metric_name": "correlation_coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "conjecture_holds": correlation_coefficient >= 0.8 and mean_difference <= 3,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 6)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)
```

```
    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(resu
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient\" first_failing_seed={f
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e7bbe8a1.py", line 84, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e7bbe8a1.py", line 52, in run_trial
    rank = formal_group_rank(f)
           ^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e7bbe8a1.py", line 31, in formal_group_
    A[i][j] = f[2**(i-1) + 2**(j-1)]
              ~^^^^^^^^^^^^^^^^^^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means it did not complete its execution to provide a correlation coefficient and mean difference for the conjecture's support condition. | next: Investigate the cause of the crash in the test code and attempt to run the test again to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17338
2	preregistration	ollama_remote	glm4:latest	0	0	9452
3	novelty	ollama_remote	glm4:latest	0	0	8216
4	novelty	ollama_remote	glm4:latest	0	0	9676
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13803
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12051
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12596
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11981
9	judge	ollama_remote	glm4:latest	0	0	17458

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 112571 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/729cd32dd1f6.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/729cd32dd1f6.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/729cd32dd1f6.tar.gz` (if generated)